

Exposition on Zero-Knowledge Proofs

Yavor Litchev

November 2023

1 Introduction

A topic in cryptography that is of extremely high interest in recent years is the research area of zero-knowledge proofs (ZKP). The idea, as is strongly suggested in the name, is that there exists protocols that if followed correctly, can allow someone to prove to another entity the validity of some fact or piece of knowledge, without revealing any (private) information on the object/subject being discussed. At a first glance, this appears to be a fantastical concept: how may one be expected to prove knowledge of something without publicly displaying the thing itself? Despite this proposition appearing to be oxymoronic, many such protocols exist, ranging in speed, efficiency, memory storage, security parameters, amount of trust in external parties, amount of external parties used for setup and/or computation, etc. In this WIM Paper, we will guide the reader through the intuition as to how ZKPs function and are constructed, and we will introduce some schemes that can scale up to provide ZKPs for fairly complex tasks.

Before we delve into how such seemingly impossible protocols function and are constructed, let us first consider the utility of such a protocol. As everyday people, we constantly have to share information in order to prove to another entity that something is true about us. For example, when inquiring about obtaining a mortgage or auto loan, one has to show, among other things, employment status, credit scores, income, etc, in order to qualify for a loan. In order to obtain certain types of medication, you have to prove that you meet certain criteria, both from specific doctor's statements and an assortment of different types of medical tests (blood tests, urine tests, etc). When qualifying for certain types of jobs, e.g. teachers, nurses, etc, one has to prove that their criminal record is clean to a degree as is required by an employer. In many of these cases, one simply has to forfeit their privacy, and simply show their personal records in order to comply with whatever procedure they are participating in. However, what if that did not necessarily have to be the case? What if one could prove that their credit score was within a certain range, without revealing one's true credit score? What if one could prove some result about their medical condition, without revealing to a hospital what the blood tests precisely measured? What if one could prove that they do not have a criminal record without showing their entire criminal history? Clearly, if such a process existed, it would receive enormous demand, as privacy and security is rising among public consciousness, and the public is searching for methods in order to better protect their own personal data. As is made abundantly clear in the prior examples, there are no shortage of use cases for these ZKP, where virtually any application where knowledge of some subject is required on some otherwise private subject could .

2 Preliminary Example: Color Blindness and Red and Green Balls

In order to demonstrate how one can prove a fact about a certain subject/object without revealing the entirety of the subject/object, consider the following example: There is a man named Victor who is red-green colorblind, in that it is impossible for him to distinguish between red and green objects. Penny, on the other hand, is not colorblind, and she wishes to demonstrate that red and green objects can be truly different. In particular, she possesses two balls: one is red, and one is

green. The balls are completely identical in every other way, so to Victor, the balls appear completely identical. Penny ponders for some time as to how she can prove to Victor that the balls are different, and then executes the following procedure:

1. Penny gives the red and green balls to Victor.
2. Victor hides the balls behind his back, draws one ball, and Penny remembers what ball it is.
3. Victor and Penny repeat the following subprocedure $n \in \mathbb{N}$ times:
 - (a) Victor hides the balls behind his back, then he shows Penny either the same ball (he did not swap) or a different ball (he swapped). Crucially, Victor decides between each action on a random basis (in that Penny has no idea what action Victor will do) Victor knows whether or not he swapped the ball.
 - (b) Victor asks Penny if he swapped the ball. If her answer corresponds with Victor's action, he continues, otherwise he aborts and says that Penny is lying, the balls are the same.
4. If Penny passes n of Victor's rounds, Victor is confident that Penny is not lying, and is convinced that the balls are indeed different.

As we can see, the above proof functions on a probabilistic basis, where a series of tests are performed with random chance introduced by Victor, as to prevent Penny from controlling or fixing the outcome of her responses. Notice that if the balls are indeed differently colored, Penny can answer Victor's questions with 100% precision. Therefore, if the balls are differently colored, Penny can succeed in this proof scheme with a probability of 1. We therefore say that this scheme satisfies *Completeness*, in that the true possession of knowledge always allows Penny to construct a valid proof in which Victor will always accept.

However, notice that if the balls are not red and green, but instead identically colored, Penny will not be unable to tell if Victor has swapped them between tests, and thus only has a 50% chance of passing each query. Therefore, in order for Penny to successfully fool Victor into believing that the balls are differently colored, she would have to successfully pass n tests, in which she has a $\frac{1}{2}^n$ chance of accomplishing. If n is sufficiently large (say, $n = 128$), Penny would have a chance lower than 1 in a trillion in fooling Victor. Thus, we say that this proofing scheme satisfies *Soundness*, in that the probability of Penny generating a fraudulent proof is negligible. Using this property, Victor can be assured that if Penny passes his tests, there is a very low chance that Penny is lying, and thus a valid proof would equate to his mind as a sound argument of knowledge.

3 Mathematical Example: Discrete Logs and Proof of Solution

The above example gives us an idea as to how to prove some fact without directly revealing the subject being concerned (Victor could not himself verify the balls being colored differently due to his colorblindness), and it allowed us to introduce key properties regarding these proofing schemes, namely that they satisfy *Completeness* (informally, Penny knowing a true fact can allow her to construct a proof that Victor will always accept) and *Soundness* (informally, Penny has a negligible chance of fooling Victor if she is lying about her knowledge, and thus Victor obtaining a valid proof means that Penny does indeed possess valid knowledge with overwhelming probability). However, although it is interesting, the above example is highly specific to a situation that is orientated in the physical world and Victor is restrained physically. We would like to be able to create ZKP not only on physical objects, but also for abstract mathematical operations, namely functions, programs, etc. How could this be accomplished?

To gain a greater understanding as to how this might be approached, we will consider the following specific problem: let $G = \mathbb{Z}_p$ be a group over a finite modular residue with prime p (typically p would be fairly large, as will be elaborated on later). Let $g \in G$ be a primitive root of G that functions as a generator, and suppose Penny wants to prove to Victor that she knows a value x such that

$g^x \equiv y \pmod{p}$, without revealing x . Note that even if y is made public, it would be computationally infeasible for someone to compute x , as that would require solving the discrete log, which under the Decisional Diffie Hellman Assumption (and the fact that p is a large prime) is an impossibility.

Analogous to how Penny proved she knew the balls were differently colored by passing Victor's random tests in the previous example, Penny leverages random queries in order to convince Victor that she does know of this special value x as follows:

1. Penny "commits" to the value y (this is analogous to Penny giving the balls to Victor in the previous example, in that she locks in her choice of what she is arguing for, and cannot change her mind later. This can be accomplished with a simple hash of the object, y , as to prevent further modification. In this example, this is probably redundant, as y is already public and Victor can see if what Penny is arguing for does not correspond to y , but in many cases parts of the object being proved might not be visible, and it is important that such a value is committed so that Victor can detect future changes. We thus introduce such a concept here.)
2. Victor and Penny repeat the following subprocedure $n \in \mathbb{N}$ times:
 - (a) Penny picks a random r , and sends Victor $s = g^r$
 - (b) Victor asks Penny to either provide r or $x + r$, with each query having a 50% chance of being randomly selected. Penny provides the value requested.
 - (c) If Victor requested r , he verifies that $g^r = s$, if Victor requested $x + r$, Victor verifies that $g^{x+r} = ys$. If the test fails, he aborts and rejects Penny's proof.
3. If Penny passes n of Victor's rounds, Victor is confident that Penny is not lying, and is convinced that Penny knows of some x such that $y = g^x$.

[4]

Let us consider a sample execution of the protocol in order to see a practical example. Suppose Penny knows that $2^8 \equiv 3 \pmod{11}$, where 2, 11, 3 are public but she wants to keep 8 private. She then sends Victor $2^3 \equiv 8 \pmod{11}$ where 2 was chosen randomly from the modular residue and is a value that only Penny knows. Victor can then either request r or $x + r$, at which point penny gives 3 or $8 + 3 = 11$ respectively. If Victor asked for r , he checks that $2^3 \equiv 8 \pmod{11}$, and if he asked for $x + r$ he checks that $2^{11} \equiv 3 * 8 \pmod{11} \equiv 2 \pmod{11}$. This is an example execution of the protocol where Victor queries 1 time (in practice, he might query hundreds of times).

Notice that in the above protocol, Victor never learns of Penny's true value, as even when requested by Victor through a test, it is masked by randomness r . Additionally, the protocol satisfies *Completeness*, since if Penny knows the true value of x , she can easily satisfy Victor's requests by providing r or $x + r$. Furthermore, the protocol satisfies *Soundness*, since if Penny does not know x (and instead only knows of the public value y), she can at best pass 1 of Victor's 2 possible queries. If she wishes to pass the test where Victor requests r , she chooses $s = g^r$. If she wishes to pass the test where victor requests $x + r$, she chooses $s = y^{-1} * g^{r'}$ and gives Victor r' (since when Victor checks, he will compute $g^{x+r} = g^{r'} = y^{-1} * g^{r'} * y = ys$, and therefore pass the query). However, Penny does not know which test Victor will choose before she locks in the value of s , and thus has only a 50% chance of passing a test. Thus, the chance that Penny will pass n tests is negligibly small.

The above example illustrates how one can perform ZKPs on mathematical functions, namely Victor asks randomized queries in order to make sure Penny does indeed know of some solution, and Penny can mask her solution with her own randomness in order to ensure zero knowledge. Although exponentiation in a finite field is a relatively specific function (as in narrow in scope) function to generate a ZKP scheme for, the ideas present in this scheme can be extrapolated to find proofs for more generic functions, e.g. proving knowledge of some polynomial, proving knowledge of addition, multiplication, general arithmetic circuits, etc.

4 Non Interaction and the Fiat-Shamir Transform

As was seen in the previous two examples of ZKPs, the general scheme followed is that Victor Performs n tests with Penny through a back and forth conversation, and each tests instills further confidence in Victor that Penny does indeed possess accurate knowledge. Although this is a very clever tactic in allowing Penny to prove her knowledge of some fact without explicitly revealing her secret, this method has some serious issues. One issue is that this back and forth conversation between Victor and Penny can be very slow. Even with the power with the modern internet, communications from one computer to another takes a few milliseconds at the fastest, and performing hundreds or thousands of tests can easily take seconds with this approach. Having a single ZKP take seconds to complete is very time consuming, especially since many algorithms require the usage of thousands of ZKPs. An even larger issue is the fact that once this ZKP is completed, only Victor will be convinced of this proof: Penny would have to perform the same procedure with every other person she wanted to be convinced by her proof. For these reasons, having an interactive protocol can be quite costly, which begs the question: could a non-interactive protocol be constructed? Could Penny produce a proof that anyone could read?

As was illustrated with the previous examples, a key component that allows the protocol to function is that Victor can challenge Penny on arbitrary values, on which Penny will not be able to prepare ahead of time to answer, and thus will have a 50% chance of failing a test if she was only posing as knowing something, when in fact she did not. In other words, the randomness component that is outside of Penny's control is what gives Victor confidence that Penny is not fooling him. Now typically, one would expect that Victor would have to be present in order to generate truly random values that Penny could not control, but what if random tests could still be generated without the presence of Victor, and without Penny controlling the randomness?

In fact, there is a way to generate random values in a deterministic way that Penny cannot control: Cryptographic Hash Functions! These functions, such as SHA-256, SHA-3, etc, are (assumed) to be one way functions, in that there is no computationally efficient way to retrieve an input value to a hash function given an output value, and therefore the output value is essentially a "random", unpredictable value. Thus, instead of relying on Victor to supply randomness for her proof, Penny can use the hash function on the current state (in the discrete-log case, the hash of the public exponentiation along with the currently used value of $s = g^r$) in order to generate random bits that would simulate Victor's random choices had he been present. This allows Penny to construct random testing herself, while allowing outsiders to be confident that Penny cannot "rig" the testing, and thus be sure that the testing is random and authentically challenges Penny. She would publish the procedure as her own proof, and a verifier can check that the random tests that Penny performed were in fact a result of a hash function, in which the inputs to the hash are clearly defined and agreed upon beforehand between Penny and potential verifiers. As a specific example, consider the following procedure Penny might employ in order to prove she knows the discrete log x to some $y = g^x$:

1. Penny "commits" to the value y
2. Penny computes a random $s = g^r$ (This randomness does not have to be outside of penny's control, as this serves as a masking value for x)
3. Penny computes $c = \text{SHA-256}(g, y, s)$ and $f = r - x * c \mod \phi(p)$
4. Penny publishes the proof as (s, f) . Anyone can verify that $s = g^f * y^c$.

[3]

The testing procedure is different than the one described earlier, in that the scheme is based on the principles behind Schorr signatures. However, the idea of the random test being supplied by a hashing function still makes the above algorithm relevant, and this transformation is used extensively in zero-knowledge proofs.

5 Generalizations: Proofs on Polynomials, Additions, Subtractions, and Perhaps Generic Circuits?

Ultimately, we would like to be able to construct ZKP in order to argue knowledge of the solution to any mathematical computation or program, while hiding the inputs that would generate the solution. In order to do this, we wish to find methods of constructing ZKP for simple functions, and then be able to generate large proofs for more complex functions based on these primitives. More specifically, we would like to be able to generate a ZKP for an arbitrary arithmetic circuit C over some finite field $\mathbb{F}_p$, where such a circuit has addition, multiplication, and replication gates. The reasoning for considering this construction is because this circuit can model any polynomial over a finite field, and arguably any mathematical function can be expressed as a polynomial through Taylor series approximations (among other methods), and finally large swaths of computational programs can be expressed as a mathematical function with inputs identical to the program's inputs. Therefore, the circuit structure described earlier can model an enormous range of computations that one would potentially want to generate a ZKP for. We would then have to generate a scheme to

First, let us consider a primitive scheme for arguing knowledge of some computation. Consider a situation where Penny would like to prove to Victor that she is aware of inputs to a circuit C that leads to a correct output. What she may do then is enumerate the computations in the circuit in a list format, commit them using a Merkle tree, and respond to an arbitrary number of Victor's random queries where she reveals some step of the computation, along with a merkleproof for said computation. This allows for Victor to be reasonably confident that Penny evaluated the circuit correctly, as an incorrect evaluation of a step by Penny can be detected by Victor with a reasonable probability. This process is also made non-interactive with the Fiat-Shamir transform (and for future reference, really any model that utilizes random queries can be made non-interactive with the Fiat-Shamir transform). The problem with this scheme is that it is firstly not zero knowledge (Penny has to reveal steps of her computation), but also the time spent on generating the proof is $O(\text{Poly}(|C|))$, and the proof length is $O(|C|)$, both of which are simply too large. Can we improve on this?

One way in which this problem is overcome is with polynomial commitments. In particular, in the Plonk scheme, Penny commits a polynomial to Victor in such a way that Victor cannot read what the original polynomial from Penny was, but can verify if Penny's computations through said polynomial are correct. The polynomial(s) that Penny commits encodes the circuit and her computations through said circuit, from which Victor can perform random queries on said polynomial in order to determine if the polynomial is in fact a correct description of Penny's computations. Furthermore, Penny can "blind" her responses to Victor's queries with a random number so as to further obfuscate the true values of her polynomial, and thus make the system zero knowledge. This scheme allows for much shorter proofs, as querying a polynomial is much more efficient than using Merkle commitments and proofs. There is too much information pertaining to the specifics of how this zk-SNARK functions to fit in this paper, and as such we encourage readers to examine Dan Boneh's 15th lecture for Stanford's CS251 course for more information. [1]

6 Zk-SNARKs

There exist many Zk-SNARKS that leverage different properties across many different domains in number theory. There are too many to discuss in a single paper, let alone in great detail, so we would like to outline specific schemes and their properties in order to demonstrate how zk-SNARKS have progressed over the years. The analysis below is outlined in Dan Boneh's 14th Lecture for the class CS 251. [2]

Firstly, Groth'16 was an important zk-SNARK protocol that caused a significant speedup of such schemes, and in a sense ignited interest in further improving zk-SNARK schemes. Incredibly, the size of the proof is constant ($O(1)$) relative to the size of an arbitrary circuit C , and the verifier time is also constant ($O(1)$) relative to C . This allows for remarkable speeds, but one of the scheme's issues, besides not being post-quantum, is that the setup process is trusted per circuit, i.e. a random

parameter has to be generated for each circuit C in order to create setup parameters for Penny to use in her ZKP generation, as well as for Victor to use to verify the proof. Maintaining the secrecy of this parameter is paramount, as the leakage of such information can allow Penny to generate false proofs. This is enormously costly for a person to generate a random parameter each time a new circuit is needed, and thus imposes serious limitations on the scalability of this protocol.

Plonk / Marlin significantly improves on this issue. Its proof generation time, as well as verification time, are both $O(1)$ (although the constant factor is slightly worse for this scheme than it is for Groth'16), but it uses a universal trusted setup. That is, it only requires true randomness one time in order to generate public parameters that can then be used for arbitrary circuits, and thus the protocol is much more scalable.

Bulletproofs is another zk-SNARK protocol that allows for Penny to prove that a number that only she knows is within a certain bounded range. This is particularly useful for financial applications where Victor would like Penny to show that some parameter is bounded and thus compliant with some rules, and in fact this protocol is used in Monero in order to obfuscate transaction amounts on the blockchain while also guaranteeing that a transaction is valid and within a certain range of acceptable transaction quantities. The proof size and verifier time are $O(\log |C|)$, and is thus much slower than the previous two schemes discussed, but an additional benefit is that it utilizes a transparent setup, i.e. no random parameters need to be generated in order to generate ZKPs. This makes the protocol highly scalable.

Finally, we will discuss STARK's. This has a proof size of $O(\log^2 |C|)$ and is thus the most space inefficient when compared to the prior 3 schemes. The verifiers time is proportional to $O(\log |C|)$, and in actuality the constant is significantly better than that for bulletproofs, so in practice this scheme has proofs that are fast to verify (marginally worse than Plonk / Marlin). It utilizes a transparent setup, allowing for great flexibility of its usage, and it is unique from the other 3 schemes in that it is considered to be post-quantum, and thus has a higher chance to be used more in the future.

The above schemes hopefully give the reader some indication as to the progress made on implementing zk-SNARK schemes, and also give insight as to what engineers consider when selecting a protocol for their chosen application. Analyzing such schemes and determining the use cases for each scheme based on their designs is a highly complex process, however this process seems to only be accelerating with the excitement and energy that has been poured into this topic over the past few years.

7 Conclusion

Zero Knowledge Proofs is an emerging field of cryptography that is rapidly expanding in the past 5 or so years. This is made apparent when one considers the explosion of research papers pertaining to this concept occurring within the last decade or so, as well as the billions of dollars being invested in private companies pertaining to zero knowledge proofs. Different schemes have different costs and benefits in terms of proof length, the time required for verification of the proof, how often private parameters need to be generated for a circuit to generate a proof (if at all), whether or not the scheme is considered to be "post quantum", etc. Currently, there does not exist a zk-SNARK scheme that is objectively better than all the other options, however when one considers the current rate of progress, it is clear that we will reach a point where ZKP will be easily accessible to the general public on any computing device, including smartphones. This is likely where future progress in ZKPs will be accentuated in: making such schemes accessible to even the smallest of computing devices. Another area for future research is to make such schemes that can function on the instruction level of CPU's and GPU's, i.e. run assembly instructions. This will allow for people to generate ZKP's for any program that is run by the modern personal computer.

Such ZKP schemes will likely be in heavy use in the near future, as people may now feasibly participate in certain computations while maintaining privacy for their personal data. This will clearly be heavily used in blockchain applications, where zk-SNARKS can make a decentralized, fully anonymous financial system a reality, but it can also be used for regular banking applications, in medical settings, tax audits, and more. The emergence of this technology seems to coincide with

the theme of how pure mathematics, although seemingly redundant and impractical at times, can be leveraged to have enormous utility for the general public. This was the case when public key cryptography was first introduced, it is also the case with ZKP's, and one can only imagine what may be possible in the future through the use of pure mathematics.

References

- [1] Dan Boneh. Building a snark. 15th University Lecture, 2023. cs251.stanford.edu/lectures/lecture15.pdf.
- [2] Dan Boneh. Using zk-snarks for privacy on the blockchain. 14th University Lecture, 2023. cs251.stanford.edu/lectures/lecture14.pdf.
- [3] Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. *Technical Report/ETH Zurich, Department of Computer Science*, 260, 1997.
- [4] JEFFREY HOFFSTEIN. *Introduction to mathematical cryptography*. SPRINGER-VERLAG NEW YORK, second edition edition, 2016.